



OPO: Data Management

Aimée Lynn Backiel
Wim Bertels
Alain Van Der Jeught

Module C: DBA & Operations

Backup & Restore

TI / BCS

2025-2026

1

SYMBOLS USED IN THIS COURSE

Symbol	Category	Typical Slide Use
	Positioning	Positioning, purpose
	Objective	Objective, goal
	Attention	Warnings, key pitfalls, critical notes
	Theory	Concept explanations, definitions
	Practice	Exercises, hands-on labs
	Clip	Demo, Clip
	Poll	Live questions / Quiz
	Informational	Neutral notes, references, reading tips. Not examinable

Slide 2

2

PURPOSE

- Gain the knowledge and skills to perform backup and restore operations in RDBMS effectively.
- Learn how to verify that backups and restores are successful.
- Understand how proper backup and restore practices protect data integrity.

Slide 3

3

OBJECTIVES

- Understand the importance of backups**
 - Explain the role of backups in data protection and disaster recovery.
 - Identify risks associated with data loss and system failures.
- Perform backup operations**
 - Execute PostgreSQL backup commands using native tools or third-party solutions.
 - Verify backup integrity.
- Perform restore operations**
 - Restore data from backups in various scenarios.

Slide 4

4

WHY BACKUPS?

- Protect business continuity and data integrity
 - Hardware failure, human error, ransomware
- Meet recovery service levels
 - Recovery Point Objective (RPO) = acceptable data loss
 - Recovery Time Objective (RTO)= acceptable downtime
- Compliance & audits
- Safe sandboxing & migrations

Slide 5

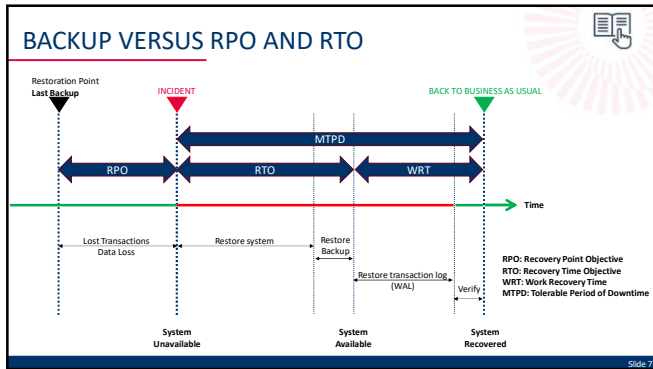
5

BACKUP TAXONOMY

 Logical dumps (pg_dump / pg_dumpall / pg_restore)	DB / schema / table granularity Portable, slower on huge DBs	
 Physical base backup (pg_basebackup)	Whole cluster snapshot Needed for Point-in-time Recovery (PITR) & physical replica seed	
 WAL archiving	Continuous change stream Combine with base backup for PITR	
 VM / Cloud snapshots	Infra-level rollback	

Slide 6

6



7

TYPICAL USE CASES

- Ad-hoc export of a table
- Full database dump pre-migration / major change
- Schema-only dump for Logical Replication (see topic on Replication) and CI/CD
- Point-in-time recovery after data corruption
- Cross-version upgrade
- DR test restore to staging
- Dev / Test DB refresh with Productive Data (be mindful of security and data privacy!)

8

PGADMIN BACKUP AND RESTORE

- **User-friendly interface** for managing logical backups and restores
- Leverages **pg_dump**, **pg_dumpall**, and **pg_restore**
- Enables easy creation of backups and database copies
- Ideal for **development and testing environments**

9

PGADMIN: LOGICAL BACKUP

Backup (Database: postgres)

General Data Options Query Options Table Options Options Objects

Filename: Backup_LocalPostgres

Format: Custom

Compression ratio: Select an item...

Encoding: Select an item...

Number of jobs: Select an item...

Role name: Select an item...

- **Backup Format Options in pgAdmin**
- **Custom**
 - Creates a compressed archive for use with **pg_restore**
 - Allows selective restore of database objects
 - **Recommended for medium to large databases**
- **Tar**
 - Generates a tar archive for use with **pg_restore**
 - **Not compressed**
 - Useful for archiving in standard format
- **Plain**
 - Creates a plain-text SQL script
 - Executable via **psql**; editable in text editors
 - **Best for small databases**
 - Not recommended for blobs
- **Directory**
 - Saves backup as a folder with one file per object
 - Includes a Table of Contents
 - Compressed by default
 - Supports parallel restore with **pg_restore**

10

PGADMIN: LOGICAL BACKUP

Backup (Database: postgres)

General Data Options Query Options Table Options Options Objects

Sections

Pre-data: ☐ Data: ☐

Post-data: ☐

Type of objects: ☐ Only schemas: ☐

Do not save: ☐

Owner: ☐ Privileges: ☐

Tablespaces: ☐ Unlogged table data: ☐

Comments: ☐ Publications: ☐

Subscriptions: ☐ Security labels: ☐

Table compressions: ☐ Table access methods: ☐

- **Sections Field Box**
 - Toggle Pre-data → right to include schema definitions (before data).
 - Toggle Data → right to include table data, large objects, and sequence values.
 - Toggle Post-data → right to include indexes, triggers, rules, and constraints.
- **Type of Objects Field Box**
 - Toggle Only data → right to back up data only.
 - Toggle Only schemas → right to back up schema objects only.
 - Toggle Blobs → left to exclude large objects from backup.
- **Do Not Save Field Box**
 - Toggle Owner → right to exclude ownership assignment commands from the backup.
 - Toggle Privileges → right to exclude access privileges commands from the backup.

11

PGADMIN: LOGICAL BACKUP

Backup (Database: postgres)

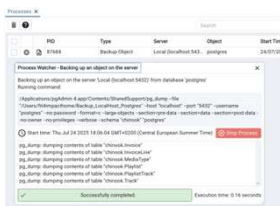
General Data Options Query Options Table Options Options Objects

Schema: ☐ Tables: ☐ Views: ☐ Schemas: ☐ Functions: ☐ Procedures: ☐ Triggers: ☐ Rules: ☐ Constraints: ☐ Indexes: ☐ Sequences: ☐ Tablespace: ☐ Unlogged table data: ☐ Comments: ☐ Publications: ☐ Subscriptions: ☐ Security labels: ☐ Table compressions: ☐ Table access methods: ☐

- **Selecting Specific Objects for Backup**
 - Use the object tree to select specific items for backup.
 - Selecting a **schema** backs up **only that schema**.
 - Selecting individual tables, views, materialized views, sequences, or foreign tables backs up only those selected objects.

12

PGADMIN: LOGICAL BACKUP



- Check log file for processing issues, if any.

Slide 13

13

PGADMIN: LOGICAL RESTORE



- Use the **Format** drop-down to choose the backup file type:
 - **Custom or Tar**: restore from custom archive files
 - **Directory**: restore from a compressed directory-format archive
- In the **Filename** field:
 - Enter the **full path** to the backup file
 - Or click the **Browse** (ellipsis) icon to select the file

Slide 14

14

PGADMIN: LOGICAL RESTORE

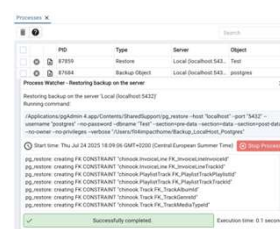


- **Sections (Content to Restore)**
 - **Pre-data** → right: restore schema definitions before data
 - **Data** → right: restore table data, blobs, and sequences
 - **Post-data** → right: restore indexes, triggers, rules, and constraints
- **Type of Objects**
 - **Only data** → right: restore **data only**
 - **Only schema** → right: restore **schema objects only**
- **Do Not Save (Exclude from Restore)**
 - **Owner** → right: skip ownership assignments
 - **Privileges** → right: skip access privilege commands

Slide 15

15

PGADMIN: LOGICAL RESTORE



- Check log file for processing issues, if any.

Slide 16

16

VERIFY BACKUPS

- **Check Backup Logs**
 - Review RDBMS/system logs for errors or warnings
- **Validate Backup Files**
 - Confirm file size, checksum, and timestamps
- **Test Restores in a Staging Environment**
 - Perform partial and full restores
 - Verify database structure, indexes, and triggers

Slide 17

17

VERIFY RESTORE

- Compare restored data against source data
- Run sample queries to ensure expected results
- Confirm user access and application connectivity
- Check performance to ensure no corruption or missing indexes

Slide 18

18

COMMON PITFALLS & FIXES

- Permission denied on dump
 - Grant rights or use superuser
- Extensions missing on restore
 - Install extensions first
- Huge dump too slow
 - Directory format + parallel jobs

Slide 19

19

BACKUP AND RESTORE

- Consult the official pgAdmin documentation for comprehensive guidance on additional backup and restore configurations.
- Demonstrating the ability to perform logical dumps (backup and restore) using pgAdmin on a PC is required for tests/exams.

Slide 20

20



Data Management

Aimée Lynn Backiel
Wim Bertels
Alain Van Der Jeught

Module C. DBA & Operations

Concurrency & Locking

TI / BCS Attribution-NonCommercial-ShareAlike 4.0 Unported Licence 2025-2026

21

SYMBOLS USED IN THIS COURSE

Symbol	Category	Typical Slide Use
	Positioning	Positioning, purpose
	Objective	Objective, goal
	Attention	Warnings, key pitfalls, critical notes
	Theory	Concept explanations, definitions
	Practice	Exercises, hands-on labs
	Clip	Demo, Clip
	Poll	Live questions / Quiz
	Informational	Neutral notes, references, reading tips. Not examinable

Slide 22

22

PURPOSE

- **Support multiple users** working on the same database at the same time.
- **Prevent conflicts** when transactions read or write the same data.
- **Maintain data integrity** in multi-user environments.
- **Locking** ensures that only safe, non-conflicting operations happen at once.
- **Minimize performance impact** while keeping data consistent.

Slide 23

23

OBJECTIVES

- **Explain** what concurrency is and why it can lead to problems in databases.
- **Identify** and **give examples** of common concurrency issues.
- **Describe** how locking works.
- **Explain and contrast** different isolation levels in ANSI SQL and PostgreSQL.
- **Analyse** and **interpret** the outcomes of two or more concurrent transactions.

Slide 24

24

PROBLEM?

- Single-user vs multi-user
- When multiple users want to read/write the same data
 - Conflicts such as inconsistent results, and partial updates if not managed properly.
 - Concurrency control is needed to handle simultaneous operations.
- Data from many tables must be consulted
 - Need for consistent reads across multiple objects.



Slide 25

25

MULTI-USER USAGE CONCURRENCY PROBLEMS

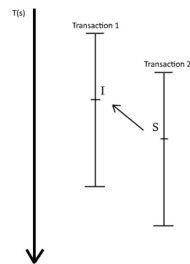
- **Dirty read (uncommitted read)**
 - A transaction reads data that was modified by another transaction but not yet committed.
 - If the other transaction rolls back, the first transaction has read data that never truly existed.
- **Nonrepeatable (or nonreproducible) read**
 - A user reads the same row twice and gets different data because another user transaction committed an update to that row between the two reads.
- **Phantom read**
 - A user re-executes the same query and gets other data due to another user inserting or deleting data in between.
- **Lost update**
 - Two users read the same data and then update it. The change made by one user is overwritten by the other user.
- **Serialisation Anomaly**
 - When several transactions run at the same time, the result can be different from what it would be if those transactions were run one after the other in order.

Slide 26

26

DIRTY READ

- A transaction reads data created by a concurrent transaction that has not yet been committed.
- If the other transaction rolls back, the first transaction has read data that was never actually saved — that's why it's called "dirty."

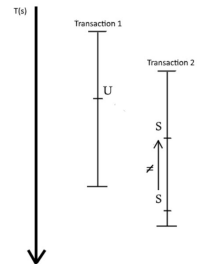


Slide 27

27

NONREPEATABLE READ

- A transaction reads the same data twice and finds that it has changed because another transaction modified and committed it between the two reads.

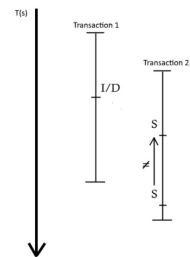


Slide 28

28

PHANTOM READ

- A transaction runs the same query again and finds new rows have appeared or some rows are missing because another transaction inserted or deleted data and committed it since the first query.

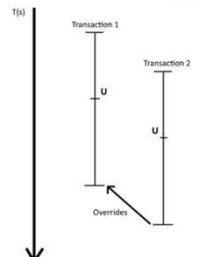


Slide 29

29

LOST UPDATE

- When two concurrent transactions update the same rows, one transaction's changes are overwritten by the other's commit, so only the last commit's changes are kept, and the earlier changes are lost.

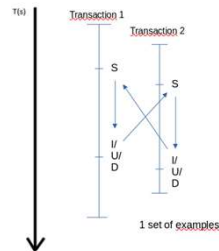


Slide 30

30

SERIALISATION ANOMALY

- A different (internal) order of overlapping transactions creates another result.



Slide 31

31

SOLUTION

- **To prevent concurrency problems:** You could run transactions one after another (**serially**), which guarantees consistency but is very slow.
- In practice, databases run many transactions **in parallel**. Hence, what techniques are used to ensure results are correct while still allowing multiple users to work simultaneously?
 - > Concurrency control techniques
 - > Isolation levels

Slide 32

32

CONCURRENCY CONTROL TECHNIQUES

- **Locking** is a mechanism that **prevents** other transactions from **modifying (and sometimes reading)** data that one transaction is currently using.
- Locks can apply to **rows, pages, or entire tables**.
- Locks are released when the transaction **commits or rolls back**.
- Lock information is managed in the database's memory.
- Different lock modes exist (e.g., **shared** for read, **exclusive** for write) and different levels of granularity.

Slide 33

33

LOCK CHALLENGES: BLOCKING

- Happens when **one transaction is waiting**; the other is **working**.
- Example
 - Transaction A locks row 1
 - Transaction B needs row 1
 - Transaction B is blocked until Transaction A commits or rolls back.
- How to reduce blocking?
 - Keep transactions short and focused.
 - Avoid locking large sets of rows unnecessarily.

Slide 34

34

LOCK CHALLENGES: DEADLOCKS

- Happens when **two or more transactions are waiting for each other's locks**.
- Example
 - Transaction A locks row 1, needs row 2.
 - Transaction B locks row 2, needs row 1.
 - Both wait forever = deadlock.
- Solution
 - The DBMS detects the deadlock automatically.
 - One transaction is aborted and frees its locks, the other transaction continues.

Slide 35

35

MULTI-VERSION CONcurrency CONTROL (MVCC)

- MVCC is a **technique used by PostgreSQL** to let multiple transactions work in parallel without blocking each other unnecessarily.
- **Instead of locking rows for reads, PostgreSQL keeps multiple versions of each row.**
- Advantage: MVCC locks acquired for querying (reading) data **do not conflict** with locks acquired for writing data, and so reading never blocks writing and writing never blocks reading.

Slide 36

36

ISOLATION LEVEL

- Isolation levels define **how much** one transaction is **protected** from seeing the effects of other concurrent transactions.
- They **control which concurrency problems** (like dirty reads, non-repeatable reads, phantom reads) are **allowed or prevented**.

Slide 37

37

ISOLATION LEVEL - ANSI SQL

Isolation Level	Definition	Allows	Prevents
Serializable	Highest isolation; transactions behave as if executed one after another.	None	Dirty Read, Non-repeatable Read, Phantom Read, Lost Update, Serialization Anomaly
Repeatable Read	Prevents changes to rows read by the transaction until it completes.	Phantom Read, Serialization Anomaly	Dirty Read, Non-repeatable Read, Lost Update
Read Committed	Each statement sees data committed at the start of that statement.	Non-repeatable Read, Phantom Read, Serialization Anomaly	Dirty Read, Lost Update
Read Uncommitted	Lowest level; May read uncommitted changes from other transactions.	Dirty Read, Non-repeatable Read, Phantom Read, Serialization Anomaly, Lost Update	None

Slide 38

38

ISOLATION LEVEL - POSTGRESQL

Isolation Level	ANSI SQL Behaviour	PostgreSQL Behaviour	Difference
Repeatable Read	Phantoms allowed	Phantoms prevented	Stricter in PostgreSQL
Read Uncommitted	Dirty reads allowed	Dirty reads prevented (same as Read Committed)	Stricter in PostgreSQL

Slide 39

39

IMPACT OF ISOLATION LEVELS

- Serializable**
 - o **Highest safety** — transactions behave as if run one at a time.
 - o **Lowest concurrency** — more waiting between users.
 - o **Slowest** for high-traffic systems.
 - o Best for **critical operations** where absolute consistency is required.
- Read Uncommitted**
 - o **Highest concurrency** — transactions wait very little for each other.
 - o **Risky** — may read changes that disappear moments later (dirty reads).
 - o In PostgreSQL, behaves exactly like Read Committed (no dirty reads)
- Avoid lengthy or long running transactions - they hold locks longer, increase wait times, and may block other users.

Slide 40

40

POSTGRESQL SYNTAX

- Check isolation level:
 - o **SHOW** default_transaction_isolation;
 - o Read committed = default isolation level
- Set isolation level
 - o **SET TRANSACTION ISOLATION LEVEL SERIALIZABLE**
 - o **SET TRANSACTION ISOLATION LEVEL REPEATABLE READ**
 - o **SET TRANSACTION ISOLATION LEVEL READ COMMITTED**
 - o **READ UNCOMMITTED** is accepted, but treated as **READ COMMITTED**

Slide 41

41

USEFULL LINKS FOR POSTGRESQL

- <http://www.postgresql.org/docs/current/static/mvcc.html>
- <http://www.postgresql.org/docs/current/static/transaction-iso.html>
- <http://www.postgresql.org/docs/current/static/sql-start-transaction.html>

Slide 42

42



OPO: Data Management

Aimée Lynn Backiel
Wim Bertels
Alain Van Der Jeught

Module C. DBA & Operations Performance: Detection & Benchmarking

TI / BCS Attribution-NonCommercial-ShareAlike 4.0 Unported Licence 2025-2026

43

SYMBOLS USED IN THIS COURSE

Symbol	Category	Typical Slide Use
	Positioning	Positioning, purpose
	Objective	Objective, goal
	Attention	Warnings, key pitfalls, critical notes
	Theory	Concept explanations, definitions
	Practice	Exercises, hands-on labs
	Clip	Demo, Clip
	Poll	Live questions / Quiz
	Informational	Neutral notes, references, reading tips. Not examinable

44

PURPOSE

- To introduce developers/DBAs to **PostgreSQL's built-in tools** for monitoring and testing performance.
- To show how to **analyze query performance (pg_stat_statements)** and **simulate workloads (pgbench)**.
- To build practical skills for **diagnosing slow queries**, **measuring throughput**, and **validating tuning changes**.

45

OBJECTIVES

- Enable and configure** the pg_stat_statements extension.
- Identify slow and high-impact queries** using pg_stat_statements.
- Reset and interpret statistics** to track workload changes.
- Understand** purpose of benchmarking and pgbench.

46

SLOW QUERIES?

- Case study: database administrator or developer wants to understand which queries are running slow.
- Solution: pg_stat_statements module

47

PG_STAT_STATEMENTS MODULE

- The pg_stat_statements module provides a means for **tracking, planning and execution statistics of all SQL statements** executed by a server.
- When pg_stat_statements is active, it **tracks statistics across all databases of the server**.
- To access and manipulate these statistics, the module provides:
 - Views: **pg_stat_statements** and **pg_stat_statements_info**,
 - Utility functions: **pg_stat_statements_reset** and **pg_stat_statements**.
 - These are not available globally but can be enabled for a specific database with **CREATE EXTENSION pg_stat_statements**.

48

HOW TO INSTALL AND ENABLE THE MODULE?

- Edit postgresql.conf to make this module available (product-specific)
- **Add** the following statements in the **config file**:
 - shared_preload_libraries = 'pg_stat_statements'
 - pg_stat_statements.max = 10000
 - pg_stat_statements.track = all
- **Restart** the server
- **In any database** where you want to use this module:
CREATE EXTENSION pg_stat_statements;



Slide 49

49

HOW TO USE PG_STAT_STATEMENTS?

- Use it to **rank queries by execution time, frequency, or rows scanned**.
- Combine with **EXPLAIN ANALYZE** to optimize problem queries.
- **Reset** periodically (per week/day) for clean analysis.



Slide 50

50

EXAMPLE USAGE SCENARIOS

- Which are the 10 slowest queries?

```
SELECT query, calls, mean_exec_time, total_exec_time
FROM pg_stat_statements
ORDER BY mean_exec_time DESC
FETCH FIRST 10 ROWS ONLY;
```



Slide 51

51

EXAMPLE USAGE SCENARIO

- Which queries consume the most database time in total (even if each call is fast)?

```
SELECT query, calls, total_exec_time, mean_exec_time
FROM pg_stat_statements
ORDER BY total_exec_time DESC
FETCH FIRST 10 ROWS ONLY;
```



Slide 52

52

USAGE?

- | | |
|----------------------------|---------------------------|
| • userid 37919 | • shared_blks_hit 0 |
| • dbid 12413 | • shared_blks_read 0 |
| • queryid 2401821812 | • shared_blks_dirtied 0 |
| • query SELECT version() | • shared_blks_written 0 |
| • calls 1 | • local_blks_hit 0 |
| • total_time 0.021 | • local_blks_read 0 |
| • min_time 0.021 | • local_blks_dirtied 0 |
| • max_time 0.021 | • local_blks_written 0 |
| • mean_time 0.021 | • temp_blks_read 0 |
| • stddev_time 0 | • temp_blks_written 0 |
| • rows 1 | • blk_read_time 0 |
| | • blk_write_time 0 |



Slide 53

53

PG_BENCH MODULE

- pgbench is a simple program for **running benchmark tests** on PostgreSQL.
- It **runs the same sequence of SQL commands over and over**, possibly in multiple concurrent database sessions, and then calculates the average transaction rate (transactions per second).
- By default, pgbench tests a scenario that is loosely based on TPC-B (a database benchmark standard) involving five SELECT, UPDATE, and INSERT commands per transaction.
- However, it is easy to test other cases by writing your own transaction script files.



Slide 54

54

REFERENCE POINTS

- Resource: pgbench
- Needs to be installed
- Example on localhost
 - \$pgbench -i probeer ("probeer" is the dbname)
 - \$ pgbench -T 120 probeer
- ..has multiple options

Slide 55

55

RESULTS

- starting vacuum...end.
- transaction type: TPC-B (sort of)
- scaling factor: 1
- query mode: simple
- number of clients: 1
- number of threads: 1
- duration: 120 s
- number of transactions actually processed: 42594
- latency average: 2.817 ms
- tps = 354.944850 (including connections establishing)
- tps = 354.961873 (excluding connections establishing)

Slide 56

56

USABILITY

- Simulations vs Reality
- Follow-up!
- A reasonable starting point
- Notas:
 - The initialisation scale factor (-s) should be at least as large as the highest number of clients you want to test (-c)
 - Autovacuum and load

Slide 57

57

REFERENCES

- <https://www.postgresql.org/docs/current/static/pgstatstatements.html>
- <https://www.postgresql.org/docs/current/static/pgbench.html>

Slide 58

58



OPO: Data Management

Aimée Lynn Backiel
Wim Bertels
Alain Van Der Jeught

Module B: Advanced SQL
Views

TI / BCS Attribution-NonCommercial-ShareAlike 4.0 Unported Licence 2025-2026

59

SYMBOLS USED IN THIS COURSE

Symbol	Category	Typical Slide Use
	Positioning	Positioning, purpose
	Objective	Objective, goal
	Attention	Warnings, key pitfalls, critical notes
	Theory	Concept explanations, definitions
	Practice	Exercises, hands-on labs
	Clip	Demo, Clip
	Poll	Live questions / Quiz
	Informational	Neutral notes, references, reading tips. Not examinable

Slide 60

60

PURPOSE

- **Simplify queries:** encapsulate commonly used instructions.
- **Reorganize data presentation:** provide logical data independence.
- **Build complex queries step-by-step:** modular “pre-programming.”
- **Enforce integrity rules:** with WITH CHECK OPTION.
- **Enhance security:** restrict access to sensitive columns/rows.

Slide 61

61

OBJECTIVES

- **Explain the purpose of SQL views**
- **Create, use, and manage views** to simplify queries, ensure integrity, and secure data access.

Slide 62

62

DEFINITION

- A **view** is a **named, virtual table** that is defined by a SELECT query.
- It does **not store data itself** (except in special cases like materialized views).
- Instead, it stores the **query definition**.
- When the view is executed, the database runs the underlying query and presents the result as if it were a regular table.

Slide 63

63

CREATING VIEWS

- **CREATE VIEW:** creates a view
- Example:

```
CREATE VIEW view_player AS
SELECT player_nr, city
FROM player
WHERE city IS NOT NULL;
```
- Views:
 - consulting and mutating
 - synonyms and comments

Slide 64

64

COLUMN NAMES OF VIEWS

- SELECT defines the column names
- But explicit definition is also possible.
- Example:

```
CREATE VIEW view_zoetermeer (snr, name, birth) AS
SELECT player_nr, last_name, birthdate
FROM player
WHERE city = 'Zoetermeer';
```
- Explicit definition is mandatory if column consists of a function or calculation.

Slide 65

65

COLUMN NAMES OF VIEWS

```
CREATE VIEW view_ages (player_nr, age) AS
SELECT player_nr, AGE(birthdate)
FROM player;

CREATE VIEW view_ages2 AS
SELECT player_nr, AGE(birthdate) AS age
FROM player;
```

Slide 66

66

WITH CHECK OPTION

- The **with check option** ensures that any data changes (INSERT or UPDATE) made through the view **must satisfy the view's defining condition**.
 - Without it: you can insert/update rows through the view that would not appear in the view afterward.
 - With it: the DBMS rejects changes that would make the row fall outside the view's condition.
- WITH CHECK OPTION, SQL checks:
 - UPDATE: adjusted rows still belong to view
 - INSERT: new row belongs to view
 - DELETE: deleted row belongs to view

Slide 67

67

WITH CHECK OPTION EXAMPLE

```
CREATE VIEW view_retiredplayers AS
SELECT player_nr, last_name
FROM player
WHERE birthdate < '1980-01-01'
WITH CHECK OPTION;
```

```
UPDATE view_retiredplayers
SET birthdate = '1981-01-01'
WHERE player_nr = 2;
```

SQL Error [44000]: ERROR: new row violates check option for view "view_retiredplayers"

Detail: Failing row contains (2, Elfring, 1981-01-01, M, 1975, Steden, 43, 3575NM, Den Haag, , R ,

player_nr	last_name	birthdate
1		
2	Elfring	1948-09-01

SQL Error [44000]: ERROR: new row violates check option for view "view_retiredplayers"

Detail: Failing row contains (2, Elfring, 1981-01-01, M, 1975, Steden, 43, 3575NM, Den Haag, , R ,

Slide 68

68

REMOVING VIEWS

- DROP VIEW:** deletes the view
- Once dropped, the view definition is deleted from the database catalog.
- The underlying base tables are **not affected**.
- CASCADE:** drop the view and any dependent objects (like other views built on top).
- RESTRICT** (default in many systems): refuse to drop if there are dependencies.

Slide 69

69

RESTRICTIONS ON MUTATING

- Possible mutations on views:
 - SELECT
 - INSERT
 - UPDATE
 - DELETE

Slide 70

70

RESTRICTIONS ON MUTATING

- But mutation is allowed only if:
 - View must be directly based on one base table (no join), indirectly can it be based on a base table via another updatable view.
- SELECT
 - may only reference columns of that one base table
 - must not contain WITH, DISTINCT, GROUP BY, HAVING, FETCH ..., or OFFSET
 - must not contain SET operators
 - must not contain an aggregate function or window function
- UPDATE: Only base table columns that appear directly in the view are updatable.
- INSERT: the view must expose all base table columns that are NOT NULL and have no default value.

Slide 71

71

PROCESSING INSTRUCTIONS ON VIEWS

- Additional step incorporating the view formula

```
SELECT player_nr
FROM player
WHERE city = 'Den Haag'
AND player_nr IN (
    SELECT player_nr
    FROM fine
);
```

```
CREATE VIEW view_expensiveplayers AS
SELECT player_nr
FROM fine
GROUP BY player_nr;
```

```
SELECT player_nr
FROM player
INNER JOIN view_expensiveplayers
USING (player_nr)
WHERE city = 'Den Haag';
```

Slide 72

72

WHEN TO USE VIEWS

- **Simplifying routine instructions**
 - Encapsulate commonly used SELECT queries so they can be reused easily.
- **Reorganisation of tables**
 - If the underlying schema changes, you can adjust the view while keeping the old queries/programs working (logical data independence).
- **Step-by-step set-up of complex queries**
 - Break down a complex SELECT into smaller views (like “pre-programming” or modular design).
- **Specifying integrity rules**
 - Use WITH CHECK OPTION to enforce conditions when inserting/updating through the view.
- **Data security**
 - Restrict access by exposing only certain rows/columns; share views instead of full tables.

73

OPO: Data Management

Aimée Lynn Backiel
Wim Bertels
Alain Van Der Jeught

Module C: DBA & Operations Schema Migration

TI / BCS Attribution-NonCommercial-ShareAlike 4.0 Unported Licence 2025-2026

74

SYMBOLS USED IN THIS COURSE

Symbol	Category	Typical Slide Use
	Positioning	Positioning, purpose
	Objective	Objective, goal
	Attention	Warnings, key pitfalls, critical notes
	Theory	Concept explanations, definitions
	Practice	Exercises, hands-on labs
	Clip	Demo, Clip
	Poll	Live questions / Quiz
	Informational	Neutral notes, references, reading tips. Not examinable

75

PURPOSE

- Ensure the database structure evolves safely with the application
- Keep data consistent while adding new features
- Minimize downtime and avoid breaking the app
- Provide a repeatable and trackable way to change the database
- Support modern development practices (CI/CD, agile delivery)

76

OBJECTIVES

- Explain what schema migrations are and why they are needed
- Perform migrations using SQL scripts with CREATE, ALTER, DROP

77

WHY DO WE NEED SCHEMA MIGRATIONS?

- Apps change → databases must change too
- Add new features (extra tables/columns)
- Keep data clean and correct
- Make queries faster

78

WHAT IS SCHEMA MIGRATION?

- A **step-by-step process** to **change the structure of a database**
- Tracks changes over time (like version history for code)
- Types of changes:
 - Add/remove tables
 - Add/remove columns
 - Add rules (constraints, keys, indexes)
- Can also “undo” a change (rollback)
- Typical DDL statements: CREATE / ALTER / DROP

Slide 79

79

WHEN DO WE APPLY SCHEMA MIGRATIONS?

- New features => need new columns/tables
- Restructuring to reduce messy data
- Improving performance (adding indexes)
- Fixing mistakes (wrong data types, missing constraints)
- New rules (e.g., GDPR or other regulations)

Slide 80

80

WHY DO WE NEED SCHEMA MIGRATIONS?

- Apps change => databases must change too
- Add new features (extra tables/columns)
- Keep data clean and correct
- Make queries faster

Slide 81

81

RISKS RELATED TO SCHEMA MIGRATIONS?

- Data loss if you delete columns/tables
- Database slowdown if changes take too long
- Different environments may be out of sync
- Hard to undo destructive changes
- App may break if it expects data that isn't ready yet

Slide 82

82

COMMON RISKS

- Data loss if you delete columns/tables
- Database slowdown if changes take too long
- Different environments may be out of sync
- Hard to undo destructive changes
- App may break if it expects data that isn't ready yet

Slide 83

83

COMMON PRACTICES

- Add new columns first => migrate data => then remove old ones
- Use **zero-downtime methods** (don't block the app)
 - E.g.: Use ADD COLUMN ... NULL instead of ADD COLUMN ... DEFAULT (because default rewrites the table).
 - Add column as NULL
 - Set default for new inserts
 - Backfill old rows late
- Test everything on staging before production
- Always backup before dangerous changes
- Deploy schema changes **before** the app depends on them

Slide 84

84

MANUAL VS TOOL SUPPORTED MIGRATION

- **Manual scripts**
 - Write SQL by hand
 - Easy but can be risky (errors, hard to track)
- **Migration tools**
 - Examples: Flyway, Liquibase, Alembic, Django
 - Keep track of versions
 - Allow automatic updates
 - Can "roll back" if something goes wrong



Slide 85

85

DDL - CREATING OBJECTS EXAMPLES

- Create table
 - **CREATE TABLE** client_test (...);
- Create index
 - **CREATE INDEX** idx_client_email **ON** client(email);



Slide 86

86

DDL – ALTERING EXISTING OBJECTS EXAMPLES

- Add column
 - **ALTER TABLE** clients **ADD COLUMN** email VARCHAR(100);
- Remove column
 - **ALTER TABLE** clients **DROP COLUMN** phone;
- Rename column
 - **ALTER TABLE** clients **RENAME COLUMN** lastname **TO** surname;
- Change column datatype, default, constraint
 - **ALTER TABLE** clients **ALTER COLUMN** email **TYPE** VARCHAR(150);
 - **ALTER TABLE** clients **ALTER COLUMN** email **SET DEFAULT** "";
 - **ALTER TABLE** clients **ALTER COLUMN** email **SET NOT NULL**;



Slide 87

87

DDL - DROPPING OBJECTS EXAMPLES

- Delete table and all data!
 - **DROP TABLE** client_test;
- Delete index
 - **DROP INDEX** idx_client_email;



Slide 88

88

DDL – CONSTRAINTS & KEYS EXAMPLES

- Add key (primary key, foreign key)
 - **ALTER TABLE** client **ADD CONSTRAINT** fk_journey **FOREIGN KEY** (journey_id) **REFERENCES** journey(journey_nr);
- Add constraint
 - **ALTER TABLE** client **ADD CONSTRAINT** check_age **CHECK** (age > 0);



Slide 89

89

DDL – REORGANISING EXAPLES

- Rename table
 - **ALTER TABLE** client **RENAME TO** passenger;
- Rename index
 - **ALTER INDEX** idx_client_email **RENAME TO** idx_passenger_email;



Slide 90

90



OPO: Data Management

Aimée Lynn Backiel
Wim Bertels
Alain Van Der Jeught

Module C: DBA & Operations Data Ethics

TI / BCS Attribution-NonCommercial-ShareAlike 4.0 Unported Licence 2025-2026

91

SYMBOLS USED IN THIS COURSE

Symbol	Category	Typical Slide Use
	Positioning	Positioning, purpose
	Objective	Objective, goal
	Attention	Warnings, key pitfalls, critical notes
	Theory	Concept explanations, definitions
	Practice	Exercises, hands-on labs
	Clip	Demo, Clip
	Poll	Live questions / Quiz
	Informational	Neutral notes, references, reading tips. Not examinable

92

PURPOSE

- Understand the **ethical responsibilities** in collecting, storing, and using data.
- Learn how ethical principles apply to **data and decision-making**.
- Explore real-world cases where ethical data practices (or the lack thereof) had significant impacts.
- Encourage critical thinking about the **social, legal, and moral implications** of data use.

93

OBJECTIVES

- Define **data ethics** and explain why it is essential in modern organizations.
- Identify **ethical risks** related to privacy, bias, transparency, and accountability.
- Apply ethical frameworks (e.g., fairness, responsibility, sustainability) to **data projects**.
- Recognize and mitigate **bias and discrimination** in algorithms and datasets.
- Understand the role of **laws and regulations** (e.g., GDPR) in guiding ethical data practices.
- Evaluate case studies to make **ethically informed decisions** in data management and analytics.

94

DATA ETHICS

- Data handling ethics are dealing with how data is procured, stored, managed, used and disposed in alignment with ethical principles such as:
 - Fairness
 - Respect
 - Responsibility
 - Integrity
 - Trust
 - Reliability
 - Quality
 - Transparency

Impact on People (internal and external) and organisation

95

DATA INTEGRITY

- What is data integrity?
 - Data integrity is a term used to describe the quality of data throughout the entire lifecycle. That means that data is valid, complete, authentic, and consistent, and it should not be changed between source and destination.
 - To fulfil these requirements, you need to have appropriate quality and risk management systems to assure the security and reliability of the data.
- Why is data integrity important?
 - Data integrity is crucial because it ensures the transparency and trustworthiness of data.
- What are common data integrity issues?
 - Most data integrity issues are related to manual tasks such as the name and time of the data creation, the instrument used to conduct the data, and other data path to other media or computers.

96

DATA INTEGRITY GDPR

- Personal Identifiable Information (PII)
- Seek advice from your Company's Data Protection Officer (DPO) when dealing with PII data, certainly when data has to be shared with external partners.

Principles relating to processing of personal data

1. Personal data shall be:
 - (a) processed lawfully, fairly and in a transparent manner in relation to the data subject (lawfulness, fairness and transparency);
 - (b) collected for specified, explicit and legitimate purposes and not further processed in a manner that is incompatible with those purposes; further processing for archiving purposes in the public interest, scientific or historical research purposes or statistical purposes shall, in accordance with article 89(2), not be considered to be incompatible with the initial purposes (purpose limitation);
 - (c) adequate, relevant and limited to what is necessary in relation to the purposes for which they are processed (data minimisation);
 - (d) accurate and where necessary kept up to date; every reasonable step must be taken to ensure that personal data that are inaccurate, having regard to the purposes for which they are processed, are erased or rectified without delay (accuracy);
 - (e) kept in a form which permits identification of data subjects for no longer than is necessary for the purposes for which the personal data are processed; personal data may be stored for longer periods insofar as the personal data will be processed solely for archiving purposes in the public interest, scientific or historical research purposes or statistical purposes in accordance with article 89(2) subject to implementation of the appropriate technical and organisational measures required by the legislator in order to safeguard the rights and freedoms of the data subject (storage limitation);
 - (f) processed in a manner that ensures appropriate security of the personal data, including protection against unauthorised or unlawful processing and against accidental loss, destruction or damage, using appropriate technical or organisational measures (integrity and confidentiality).
2. The controller shall be responsible for and be able to demonstrate compliance with paragraph 1 ('accountability').

Source: <https://gdpr-info.eu/art-5-gdpr/>

Slide 97

97

FINANCIAL DATA / SOX PRINCIPLES

- The Sarbanes-Oxley Act (SOX) defines the requirements for the integrity of source data related to financial transactions and disclosures.
- SOX Section 404 requires implementation of technical controls and continuous access auditing to assure the reliability of data related to financial transactions.
- Data Accuracy and Integrity:
 - Data accuracy is paramount in the realm of financial reporting, and data governance serves as the bedrock for ensuring these critical attributes. By aligning with the SOX, emphasis on reliable financial reporting, data governance becomes a proactive mechanism to identify and rectify inaccuracies or inconsistencies in financial data. This not only enhances the precision of financial statements but also instills confidence in stakeholders regarding the authenticity and reliability of the information presented.
- Data Security and Confidentiality:
 - Safeguarding sensitive financial information is a non-negotiable aspect of corporate responsibility, and SOX mandates precisely that. Data governance plays a pivotal role in fortifying **data security** and confidentiality by implementing a multifaceted approach. This includes defining and enforcing stringent access controls to ensure that only authorized personnel have access to sensitive financial data.

Slide 98

98

ALCOA++ PRINCIPLES

- ALCOA++ defines a framework to achieve data integrity, which is especially important for regulated industries such as life sciences
 - Attributable: Record the person or system that performed an activity.
 - Legible: Data must be readable throughout the entire lifecycle of the record.
 - Contemporaneous: Documented at the time of the activity.
 - Original: Record is original or certified true copy.
 - Accurate: No errors of editing without documented amendments.
- In addition to the core ALCOA principles, data should also be:
 - Complete: All data, including all tests, repetition, or reanalysis, must be documented.
 - Consistent: All components of the analysis, such as a sequence of events, are followed.
 - Enduring: A sustainable record (systematically documented) in a validated system.
 - Available: Data can be accessed for review, audit, or inspection over its lifetime.
 - Traceability: Data is traceable throughout the process and life cycle, including change

Slide 99

99

FAIR PRINCIPLES

- Set of guidelines to improve the Findability, Accessibility, Interoperability, and Reusability of digital assets .
- The principles emphasise machine-actionability (i.e., the capacity of computational systems to find, access, interoperate, and reuse data with none or minimal human intervention) because humans increasingly rely on computational support to deal with data as a result of the increase in volume, complexity, and creation speed of data.

Slide 100

100

FAIR PRINCIPLES

- Findable means that either people or computers can find your data. This applies to data as well as metadata, which essentially describes other data.
 - F1. (meta)data are assigned a globally unique and persistent identifier
 - F2. data are described with rich metadata (defined by R1 below)
 - F3. metadata clearly and explicitly include the identifier of the data it describes
 - F4. (meta)data are registered or indexed in a searchable resource

Slide 101

101

FAIR PRINCIPLES

- Accessible data does not mean that your data must be open for everyone to access, but rather that adequate authentication is needed to gain access to it.
 - A1. (meta)data are retrievable by their identifier using a standardized communications protocol
 - A1.1. the protocol is open, free, and universally implementable
 - A1.2. the protocol allows for an authentication and authorization procedure, where necessary
 - A2. metadata are accessible, even when the data are no longer available

Slide 102

102

FAIR PRINCIPLES

- Interoperability of data means that both people and machines can recognize and use certain data. Standardization and usage of recognizable file formats are essential to achieve this.
 - 1. (meta)data use a formal, accessible, shared, and broadly applicable language for knowledge representation.
 - 2. (meta)data use vocabularies that follow FAIR principles
 - 3. (meta)data include qualified references to other (meta)data

Slide 103

103

FAIR PRINCIPLES

- Reusable data is the data that other people can easily use to understand and recreate the experiment.
 - R1. meta(data) are richly described with a plurality of accurate and relevant attributes
 - R1.1. (meta)data are released with a clear and accessible data usage license
 - R1.2. (meta)data are associated with detailed provenance
 - R1.3. (meta)data meet domain-relevant community standards

Slide 104

104

EU AI ACT

- The **AI Act** (formally *Regulation (EU) 2024/1689*) is the European Union’s first comprehensive law governing artificial intelligence (AI). It is part of the EU’s strategy to promote “*trustworthy AI*” and seeks to ensure that AI systems placed on the EU market are safe, respect fundamental rights and are overseen by people rather than by fully automated processes . The regulation entered into force on **1 August 2024** and will be applied gradually through phases that run from February 2025 to August 2027

<https://digital-strategy.ec.europa.eu/en/policies/european-approach-artificial-intelligence>

Slide 105

105

EU AI ACT

- Risk-based classification**
- The Act takes a risk-based approach, categorising AI systems into four levels:
- Unacceptable-risk AI** – systems that threaten people’s rights or safety are banned. Prohibited practices include manipulative or deceptive AI that distorts behaviour, exploiting vulnerabilities, social scoring, AI-based prediction of criminal offences, untargeted scraping for facial-recognition databases, emotion-recognition at work or school, biometric categorisation of sensitive traits, and real-time remote biometric identification in public spaces .
- High-risk AI** – AI used in critical areas (e.g., safety components in transport or energy, education admission, robot-assisted surgery, hiring and workplace management, credit scoring and access to essential services, certain law-enforcement and border-control applications, and judicial processes) is subject to strict obligations . Providers must conduct risk management, use high-quality and representative datasets, log system activities, produce technical documentation, provide clear instructions to users, ensure human oversight, and design for robustness, cybersecurity and accuracy .
- Limited-risk AI** – systems with mainly transparency risks (such as chatbots or generative-AI models) must inform users that they are interacting with AI and label AI-generated content like deep fakes . Providers of generative models must publish summaries of the data used for training and comply with EU copyright rules .
- Minimal- or no-risk AI** – most common AI applications (e.g., spam filters or AI-enabled video games) fall into this category and are not regulated

Slide 106

106



OPO: Data Management

Aimée Lynn Backiel
Wim Bertels
Alain Van Der Jeught

Module C. DBA & Operations
High Availability

TI / BCS
Attribution-NonCommercial-ShareAlike 4.0 Unported Licence
2025-2026

107

SYMBOLS USED IN THIS COURSE

Symbol	Category	Typical Slide Use
	Positioning	Positioning, purpose
	Objective	Objective, goal
	Attention	Warnings, key pitfalls, critical notes
	Theory	Concept explanations, definitions
	Practice	Exercises, hands-on labs
	Clip	Demo, Clip
	Poll	Live questions / Quiz
	Informational	Neutral notes, references, reading tips. Not examinable

Slide 108

108

PURPOSE

- Understand why **high availability (HA)** is critical for modern databases.
- Learn the architectural principles behind **distributed databases**.
- Explore trade-offs through the **CAP and PACELC theorems**.
- Examine HA techniques.

Slide 109

109

OBJECTIVES

- Explain the concepts of **high availability** in database systems.
- Describe the **CAP theorem** and **PACELC theorem**, and analyze trade-offs in distributed systems.
- Compare and contrast different **replication strategies**.
- Identify **use cases and limitations** of distributed DBMS approaches
- Apply **logical replication in practice**.

Slide 110

110

HIGH AVAILABILITY

- **Definition:** The ability of a system to remain accessible and operational even in the face of failures.
- **Goal:** Minimize downtime (measured in "nines" – e.g., 99.99% uptime).
- **Techniques (DBMS-dependent):**
 - **Replication** (standby nodes)
 - **Partitioning/Sharding** (Data is distributed across nodes)
 - **Failover** (automatic switchover on failure)
 - **Load balancing** (spread traffic across nodes)
 - **Monitoring & recovery**
 - ...

Slide 111

111

DISTRIBUTED DATABASE

- A **distributed database** is a collection of data that is stored across **multiple computers (nodes/servers)** but appears to users as **one single database**.
- The nodes are connected by a network.
- Data may be replicated (copies) or partitioned (shards).
- The system manages distribution **transparently** (so users query it with normal SQL/queries).
- A **distributed database system** is about the **architecture, not the data model**.

Slide 112

112

TYPES OF DISTRIBUTED DATABASES

- **Homogeneous**
 - All nodes run the same DBMS
- **Heterogeneous**
 - Different DBMSs across nodes
- **Replication-based**
 - Same data copied across nodes for availability (master/replica).
- **Partitioned (sharded)**
 - Data split across nodes, each node holds a subset.

Slide 113

113

EXAMPLE DISTRIBUTED DATABASE SYSTEMS

- **Relational Distributed DBMS**
 - **Postgres + Citus extension** – Turns PostgreSQL into a distributed/sharded system for multi-tenant or analytics workloads.
 - **Google Spanner** – Globally distributed SQL database, strong consistency, used internally at Google and via Google Cloud.
 - **CockroachDB** – Open-source, relational, Postgres-compatible, strongly consistent across regions.
- **NoSQL Distributed DBMS**
 - **Apache Cassandra** – Wide-column store, used by Netflix, Apple.
 - **Amazon DynamoDB** – Fully managed key-value/document store, high availability, eventually consistent by default.
 - **MongoDB (clustered / sharded)** – Document store, can be deployed as a distributed system with replica sets + sharding.
 - **HBase** – Column-oriented distributed database built on top of Hadoop.

Slide 114

114

CAP THEOREM

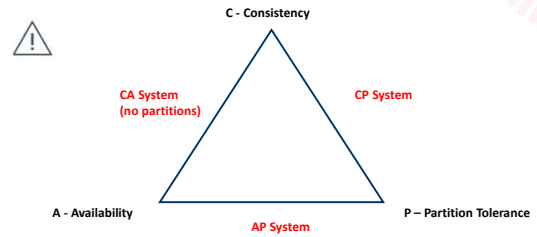
- **CAP Theorem:** in the presence of a **network partition**, a **distributed database system** can guarantee **at most two** of the following three properties:
 - **C - Consistency:**
 - all nodes in the system see the same data at the same time.
 - **A - Availability:**
 - every request gets a response back.
 - **P - Partition tolerant:**
 - if a node fails, the system continues to function.

In practice partition tolerant is non-negotiable in distributed systems.
Trade-off is **Consistency vs Availability**

Slide 115

115

CAP THEOREM



Slide 116

116

CAP THEOREM

- **CP Systems** (Consistency + Partition Tolerance)
 - Guarantee that all nodes agree on the same data (Consistency).
 - Will reject or delay requests during a partition instead of serving stale data.
 - Trade-off: Less availability — some requests may fail or hang during network issues.
- **AP Systems** (Availability + Partition Tolerance)
 - Always respond to queries, even during partitions (Availability).
 - Allow temporary differences between nodes (eventual consistency).
 - Trade-off: Less strict consistency — you might read stale data.
- **CA Systems** (Consistency + Availability)
 - Theoretically possible only if there are no partitions (i.e., in a single-node or fully reliable network).
 - In distributed, real-world systems, partitions happen → so CA alone is not achievable.

Slide 117

117

PACELC THEOREM

- **Refinement of CAP theorem**
 - CAP: During a **Partition (P)**, a system must choose between **Availability (A)** and **Consistency (C)**.
 - PACELC adds: **Else (E)**, when the system is running normally without partitions, it must still choose between **Latency (L)** and **Consistency (C)**.
- CAP only considers what happens **when partitions occur**, but in real distributed systems, even without failures, you must choose:
 - **Low latency** (fast responses, maybe stale data).
 - **Strong consistency** (slower, but guaranteed up-to-date).

Distributed databases always face trade-offs — not just in failure, but also in normal operation.

Slide 118

118

DISTRIBUTED DATABASES PROS AND CONS

- **Pros**
 - **Scalability** → handle more data and queries by adding nodes.
 - **Fault tolerance** → if one node fails, others continue.
 - **Geographic distribution** → place data near users for faster access.
- **Cons**
 - **Consistency vs Availability** → CAP theorem trade-offs.
 - **Complex transactions** → harder to support ACID across nodes.
 - **Latency** → cross-node communication can be slow.
 - **Complexity** → setup, monitoring, and recovery are more difficult.

Slide 119

119

REPLICATION

- **What:** Keep copies of your data on multiple servers.
- **Why for DBs:** If the primary database fails, a replica can take over.
- **Postgres example:** Streaming replication (synchronous or asynchronous).

Slide 120

120

PHYSICAL REPLICATON

- **What:** Copies the **binary WAL (Write-Ahead Log) files** from the primary to the standby.
- **How:** The standby replays the WAL exactly, so it becomes a **block-for-block copy** of the primary.
- **Features:**
 - Very fast, low overhead.
 - Standby is **read-only** (Hot Standby allows read queries).
 - Can be **synchronous** or **asynchronous**.
- **Use case:** High Availability (HA), disaster recovery.

Slide 121

121

STREAMING REPLICATON

- **What:** Instead of shipping WAL files in bulk, the primary streams WAL **in real-time** to replicas.
- **How:** Standbys connect via a replication protocol (walreceiver process).
- **Features:**
 - More up-to-date than WAL file shipping (near real-time replication).
 - Can combine with **archiving** for durability.
 - Supports cascading replication (replica streams to another replica).
- **Use case:** Keeping hot standby replicas in sync with minimal lag.

Slide 122

122

LOGICAL REPLICATON

- **What:** Replicates **data changes (INSERT, UPDATE, DELETE)** at the **row level**, not WAL blocks.
- **How:** Uses **publish/subscribe** model:
 - Publisher = source DB
 - Subscriber = target DB
- **Features:**
 - Flexible: can replicate **specific tables/columns**, not entire database.
 - Supports replication across **different Postgres versions**.
 - Data on subscriber can be queried/modified independently.
 - More overhead than physical replication.
- **Use case:**
 - Multi-master setups, data migrations, partial replication, cloud migrations.

Slide 123

123

LOGICAL REPLICATON SETUP STEPS

- **Steps:**
 - **Configure publisher (source DB)** in config file
 - **Create a publication on the publisher (source DB)**
 - **Configure subscriber (target DB)**
 - **Create the subscription (target DB)**
 - **Test the replication (source and target DB)**
- **Important**
 - Logical replication replicates **INSERT/UPDATE/DELETE**, but not schema changes (DDL).
 - You can replicate **selected tables only**, unlike physical replication which is cluster-wide.
 - The subscriber's table must have the **same schema** (columns & types).
 - Subscriber tables are **writable**.

Slide 124

124

LOGICAL REPLICATON EXAMPLE

- In `postgresql.conf` of `space_travel` (on source DB):
`wal_level = logical`
`max_wal_senders = 10`
`max_replication_slots = 10`
- **Create a publication on the publisher (on source DB) with a user with replication rights on the published tables**


```
CREATE ROLE replicator WITH LOGIN PASSWORD 'secret';
GRANT USAGE ON SCHEMA space_travel TO replicator;
GRANT SELECT ON ALL TABLES IN SCHEMA space_travel TO replicator;

CREATE PUBLICATION celestial_object_pub
FOR TABLE space_travel_source.celestial_object;
```

Slide 125

125

LOGICAL REPLICATON EXAMPLE

- **Configure subscriber (on target DB)**
`CREATE SCHEMA / TABLE celestial_object` or `RESTORE` source DB schema for table (**NO DATA restore!**) / **Table must have PK**
- **Create the subscription (on target DB)**

```
CREATE SUBSCRIPTION celestial_object_sub
CONNECTION 'host=localhost port=5432 dbname=space_travel user=replicator password=secret'
PUBLICATION celestial_object_pub;
```
- **Test the replication**
Insert, Update or Delete row in Source DB table
Check whether inserted, updated or deleted row is also maintained in Target DB table.

Slide 126

126

OVERVIEW WITHIN PLANET PG

Feature	Shared Disk Repl.	File System Repl.	Write-Ahead Log Shipping	Logical Repl.	Trigger-Based Repl.	SQL Repl. Middle-ware	Async. MM Repl.	Sync. MM Repl.
Popular examples	NAS	DRBD	built-in streaming repl.	built-in logical repl. pglogical	Logical Decoding	pgpool-II	Bucardo	
Comm. method	shared disk	disk blocks	WAL	logical decoding	table rows	SQL	table rows	table rows and row locks
No special hardware required		*	*	*	*	*	*	*
Allows multiple primary servers				*		*	*	*
No overhead on primary	*		*		*	*	*	*
No waiting for multiple servers	*		with sync off	with sync off	*	*	*	*
Primary failure will never lose data	*	*	with sync on	with sync on	*	*	*	*
Replicas accept read-only queries			with hot standby	*	*	*	*	*
Per-table granularity				*	*	*	*	*
No conflict resolution necessary	*	*	*	*	*	*	*	*

Slide 127

127

OTHER: NOT BUILT IN REPLICATION

- <https://www.symmetricds.org/>

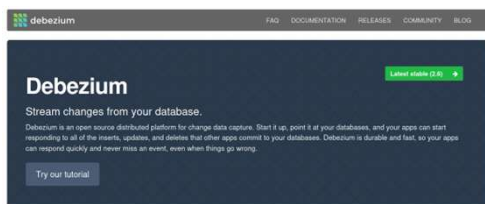


Slide 128

128

OTHER: NOT BUILT IN REPLICATION

- <https://debezium.io/>



Slide 129

129

PARTITIONING/SHARDING

- **What**
 - **Partitioning:** Splitting a large table into smaller, more manageable parts (partitions).
 - **Sharding:** Distributing partitions across **multiple servers/nodes** to scale out and reduce single points of failure.
- **Why for Ds?**
 - **Scalability** – spreads workload across nodes.
 - **High Availability** – node failure only affects one shard, not the whole DB.
 - **Performance** – queries target smaller partitions instead of entire tables.
- **Postgres example**
 - **Native Partitioning:** Declarative partitioning (e.g., by range, list, or hash).
 - **Citus Extension:** Turns Postgres into a **distributed, sharded database**.

Slide 130

130

FAILOVER (AUTOMATIC SWITCHOVER)

- **What:** Automatically promote a standby node to primary if the main one fails.
- **Why for DBs:** Ensures minimal downtime and manual intervention.
- **Postgres example:** Patroni, repmgr, pgpool-II manage failover.

Slide 131

131

LOAD BALANCING

- **What:** Distribute read/write queries across multiple nodes.
- **Why for DBs:**
 - Improves performance (reads go to replicas).
 - Prevents overloading a single primary.
- **Postgres example:** pgpool-II, HAProxy can direct queries to replicas.

Slide 132

132

MONITORING & RECOVERY

- **What:** Continuous health checks, log collection, and automated repair.
- **Why for DBs:** Early detection of failures → trigger failover/alerts.
- **Postgres example:** Tools like Prometheus + Grafana, pg_stat_monitor, Nagios.

Slide 133

133

BACKUP VS REPLICATION VS FDW (SQL/MED)

- **Backup** = recovery tool.
- **Replication** = high availability + performance.
- **FDW (SQL/MED)** = integration across systems.
- **Analogy**
 - **Backup** = Taking a **photo** of your database for safekeeping.
 - **Replication** = Having a **mirror** of your database, always kept up to date.
 - **FDW** = Opening a **window** into another system so you can look inside and query its data.



Slide 134

134